

Lab Assignment # 17.1

Program : B. Tech (CSE)
Specialization
Course Title : AI Assisted coding
Course Code
Semester II
Academic Session : 2025-2026
Name of Student : A Vamshi Krishna
Enrollment No. : 2403A51L47
Batch No. : 52
Date :31-03-2026

Task 1 - Customer Data Cleaning

Task:

Use AI to generate a Python script for cleaning a **customer dataset**.

Instructions:

- Handle missing values in columns (age, income, city).
- Remove duplicate records.
- Standardize text values (e.g., city names in lowercase).
- Encode categorical variables (gender, city).

Code:

```
File Edit View Run Kernel Settings Help
B + X [Icons] C [Icons]
JupyterLab 0 Python (conda env:base) • Q [Icons]

[3]: import pandas as pd
    from sklearn.preprocessing import LabelEncoder

    # Load dataset
    data = pd.read_csv("shopping_trends.csv")

    print("Original Data:")
    print(data.head())

    ## -----
    ## Handle Missing Values (FIXED)
    ## -----
    if 'Age' in data.columns:
        data['Age'] = data['Age'].fillna(data['Age'].mean())

    if 'Purchase Amount (USO)' in data.columns:
        data['Purchase Amount (USO)'] = data['Purchase Amount (USO)'].fillna(
            data['Purchase Amount (USD)'].mean())

    if 'Location' in data.columns:
        data['Location'] = data['Location'].fillna(data['Location'].mode()[0])

    ## -----
    ## Remove Duplicates
    ## -----
    data = data.drop_duplicates()

    ## -----
    ## Standardize Text
    ## -----
    if 'Location' in data.columns:
        data['Location'] = data['Location'].str.lower()

    if 'Gender' in data.columns:
        data['Gender'] = data['Gender'].str.lower()

    ## -----
```

```

#####
#4.Encode Categorical Variables
#####
le = LabelEncoder()

if 'Gender' in data.columns:
    data['Gender'] = le.fit_transform(data['Gender'])

if 'Location' in data.columns:
    data['Location'] = le.fit_transform(data['Location'])

#####
#Final Output
#####
print("\nCleaned Data:")
print(data.head())

data.to_csv("cleaned_customer_data.csv", index=False)

```

Output:

Original Data:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (US\$)
0	1	55	Male	Blouse	Clothing	53
1	2	19	Male	Sweater	Clothing	64
2	3	50	Male	Jeans	Clothing	73
3	4	21	Male	Sandals	Footwear	90
4	5	45	Male	Blouse	Clothing	49

	Location	Size	Color	Season	Review Rating	Subscription Status
0	Kentucky	L	Gray	Winter	3.1	Yes
1	Maine	L	Maroon	Winter	3.1	Yes
2	Massachusetts	5	Maroon	Spring	3.1	Yes
3	Rhode Island	M	Maroon	Spring	3.5	Yes
4	Oregon	M	Turquoise	Spring	2.7	Yes

	Payment Method	Shipping Type	Discount Applied	Promo Code	Used
0	Credit Card	Express	Yes		Yes
1	Bank Transfer	Express	Yes		Yes
2	Cash	Free Shipping	Yes		Yes
3	PayPal	Next Day Air	Yes		Yes
4	Cash	Free Shipping	Yes		Yes

	Previous Purchases	Preferred Payment Method	Frequency of Purchases
0	14	Venmo	Fortnightly
1	2	Cash	Fortnightly
2	23	Credit Card	Weekly
3	49	PayPal	Weekly
4	31	PayPal	Annually

Cleaned Data:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (US\$)
0	1	55	1	Blouse	Clothing	53
1	2	19		Sweater	Clothing	64
2	3	50		Jeans	Clothing	73
3	4	21		Sandals	Footwear	90
4	5	45		Blouse	Clothing	49

Cleaned Data:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (US\$)
	1	55	1	Blouse	Clothing	53
	2	19		Sweater	Clothing	64
	3	50		Jeans	Clothing	73
	4	21		Sandals	Footwear	90
4	5	45		Blouse	Clothing	49

	Location	Size	Color	Season	Review Rating	Subscription Status
	16	L	Gray	Winter	3.1	Yes
	18	L	Maroon	Winter	3.1	Yes
	20	5	Maroon	Spring	3.1	Yes
	38	M	Maroon	Spring	3.5	Yes
4	36	M	Turquoise	Spring	2.7	Yes

	Payment Method	Shipping Type	Discount Applied	Promo Code	Used
	Credit Card	Express	Yes		Yes
	Bank Transfer	Express	Yes		Yes
	Cash	Free Shipping	Yes		Yes
	PayPal	Next Day Air	Yes		Yes
4	Cash	free Shipping	Yes		Yes

	Previous Purchases	Preferred Payment Method	frequency of Purchases
	14	Venmo	Fortnightly
	2	Cash	fortnightly
	23	Credit Card	Weekly
	49	PayPal	Weekly
4	31	PayPal	Annually

Task 2 - Sales Data Preprocessing

Task:

Preprocess **sales transaction data** using AI assistance.

Instructions:

- Convert date columns into datetime format.
- Create new features (month, year, day of week).
- Normalize sales amount column.
- Detect and handle outliers in dataset.

Code:

```
[5]: import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

data; pd.read_csv("sales_data.csv")

print("Columns:", data.columns)

##-----
#Auto detect date column
##-----
date_col; None
for col in data.columns:
    if 'date' in col.lower():
        date_col; col
        break

# Convert to datetime
data[date_col]; pd.to_datetime(data[date_col])

# Create features
data['year']; data[date_col].dt.year
data['month']; data[date_col].dt.month
data['day_of_week']; data[date_col].dt.day_name()

##-----
#Detect sales column
##-----
sales_col = None
for col in data.columns:
    if 'sale' in col.lower():
        sales_col = col
        break

# Normalize
scaler; MinMaxScaler()
data['sales_normalized']; scaler.fit_transform(data[sales_col])

##-----
#Outlier Removal (IQR)
##-----
Q1; data[sales_col].quantile(0.25)
Q3; data[sales_col].quantile(0.75)
IQR; Q3 - Q1

lower; Q1 - 1.5 * IQR
upper; Q3 + 1.5 * IQR

data; data[(data[sales_col] >= lower) & (data[sales_col] <= upper)]

# Output
print("\nProcessed Data:")
print(data.head())

data.to_csv("processed_sales_data.csv", index=False)
```

Output:

```
Columns: Index(['Product_ID', 'Sale_Date', 'Sales_Rep', 'Region', 'Sales_Amount',
               'Quantity_Sold', 'Product_Category', 'Unit_Cost', 'Unit_Price',
               'Customer_Type', 'Discount', 'Payment_Method', 'Sales_Channel',
               'Region_and_Sales_Rep'],
              dtype='object')
```

Processed Data:

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold
0	1052	2023-02-03	Bob	North	5053.97	18
1	1093	2023-04-21	Bob	West	4384.02	17
2	1015	2023-09-21	David	South	4631.23	30
3	1072	2023-08-24	Bob	South	2167.94	39
4	1061	2023-03-24	Charlie	East	3750.20	13

	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount
0	Furniture	152.75	267.22	Returning	0.09
1	Furniture	3816.39	4209.44	Returning	0.11
2	Food	261.56	371.40	Returning	0.20
3	Clothing	4330.03	4467.75	New	0.02
4	Electronics	637.37	692.71	New	0.08

	Payment_Method	Sales_Channel	Region_and_Sales_Rep	year	month	day_of_week
0	Cash	Online	North-Bob	2023	2	Friday
1	Cash	Retail	West-Bob	2023	4	Friday
2	Bank Transfer	Retail	South-David	2023	9	Thursday
3	Credit Card	Retail	South-Bob	2023	8	Thursday
4	Credit Card	Online	East-Charlie	2023	3	Friday

	sales_normalized
0	0.090411
1	0.301370
2	0.720548
3	0.643836
4	0.224658

Task 3 - Healthcare Data Preparation

Task:

Clean and preprocess a **healthcare dataset** using AI scripts.

Instructions:

- Handle missing values in blood_pressure, cholesterol.
- Encode categorical features
- Scale numerical features (min-max or standard scaling).
- Split dataset into training and testing sets.

Code:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from sklearn.model_selection import train_test_split

# Load dataset
data = pd.read_csv("dirty_v3_path.csv")

print("Original Data:")
print(data.head())

## -----
## Check columns (IMPORTANT)
## -----
print("\nColumns:", data.columns)

## -----
## Handle Missing Values
## -----
# Auto-detect columns
for col in data.columns:
    if 'blood' in col.lower():
        data[col] = data[col].fillna(data[col].mean())

    if 'chol' in col.lower():
        data[col] = data[col].fillna(data[col].mean())

## -----
# 2. Encode Categorical Features
## -----
le = LabelEncoder()

for col in data.columns:
    if data[col].dtype == 'object':
        data[col] = le.fit_transform(data[col])
```

```

#-----
#3. Scale Numerical Features (Min-Max)
#-----
scaler= MinMaxScaler()

num_cols = data.select_dtypes(include=['int64', 'float64']).columns
data[num_cols] = scaler.fit_transform(data[num_cols])

#-----
#4. Split Data (Train/Test)
#-----
#Assume last column is target (you can change if needed)
X = data.Hoc[:, :-1]
y = data.iloc[:, -1]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

#-----
#Final Output
#-----
print("\nProcessed Data:")
print(data.head())

print("\nTrain shape:", X_train.shape)
print("\nTest shape:", X_test.shape)

# Save file
data.to_csv("processed_healthcare_data.csv", index=False)

```

Output:

Original. Data :

	Age	Gender	Medical. Condition	Glucose	Blood Pressure	BMI
0	45.0	Male	Healthy	97.0	127.27	28.96
1	22.0	Male	Healthy	71.58	113.27	26.29
2	50.0	NaN	Asthma	95.24	NaN	22.53
3	57.0	NaN	Obesity	NaN	130.53	38.47
4	66.0	Female	Hypertension	95.15	178.17	31.12

	Oxygen Saturation	LengthOfStay	Cholesterol	Triglycerides	HbA1c
0	96.04	6	231.88	210.56	7.61
1	97.54	2	165.57	129.41	4.91
2	90.31	2	214.94	165.35	5.60
3	96.60	5	197.71	182.13	6.92
4	94.90	4	259.53	115.85	5.98

	Smoking	Alcohol	Physical Activity	Diet Score	Family History
0	0	0	-0.20	3.54	0
1	0	0	0.12	5.90	0
2	0	0	5.01	4.65	1
3	0	0	3.16	3.37	0
4	0	1	3.56	3.40	0

	Stress Level	Sleep Hours	random_notes	noise_col
0	5.07	6.05	lorem	-137.057211
1	5.87	7.72	ipsum	-11.230610
2	3.09	4.82	ipsum	98.331195
3	3.01	5.33	lorem	44.187175
4	6.38	6.64	lorem	44.831426

Columns: Index('Age', 'Gender', 'Medical Condition', 'Glucose', 'Blood Pressure', 'BMI', 'Oxygen Saturation', 'LengthOfStay', 'Cholesterol', 'Triglycerides', 'HbA1c', 'Smoking', 'Alcohol', 'Physical Activity', 'Diet Score', 'Family History', 'random_notes', 'noise_col', 1.

Precessed Data:

	Age	Gender	Medical Condition	Glucose	Blood Pressure	BMI
0	0.455696	0.5	0.428571	0.391428	0.401144	0.431680
1	0.151899	0.5	0.571429	0.171904	0.256540	0.378609
2	0.506329	1.0	0.142857	0.251249	0.435226	0.302155
3	0.594937	1.0	0.857143	NaN	0.369988	0.626271
4	0.708861	0.0	0.714286	0.250947	0.683121	0.476820

	Oxygen Saturation	LengthOfStay	Cholesterol	Triglycerides	HbA1c
0	0.670348	0.277778	0.518390	0.524877	0.476872
1	0.705592	0.055556	0.265915	0.342102	0.179515
2	0.535714	0.055556	0.453891	0.423050	0.255507
3	0.683506	0.222222	0.388288	0.460844	0.400881
4	0.643562	0.166667	0.623667	0.311561	0.297357

	Smoking	Alcohol	Physical Activity	Diet Score	Family History
0	0.0	0.0	0.216283	0.383056	0.0
1	0.0	0.0	0.733375	0.553946	0.0
2	0.0	0.0	0.540087	0.463432	1.0
3	0.0	0.0	0.425109	0.370746	0.0
4	0.0	1.0	0.449969	0.372918	0.0

	Stress Level	Sleep Hours	random_notes	noise_col
0	0.419788	0.509132	1.000000	0.312605
1	0.464505	0.699772	0.666667	0.455579
2	0.309111	0.368721	0.666667	0.580072
3	0.304639	0.426941	1.000000	0.518549
4	0.493013	0.576484	1.000000	0.519281

Train shape: (24000, 19)

Test shape: (6000, 19)

Task 4 - Employee Salary Data Preprocessing

Task:

Clean and preprocess an employee salary dataset using AI scripts.

Instructions:

- Handle missing values
- Detect and remove salary outliers.
- Standardize department names (lowercase).
- Apply label encoding to job location, department.

Code:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Load dataset
data= pd.read_csv("Employee_Salary_Dataset.csv")

print("Original Data:")
print(data.head())

# -----
# Check columns (IMPORTANT)
# -----
print("\n(columns: ", data.columns)

# -----
# 1. Handle Missing Values
# -----
for col in data.columns:
    if data[col].dtype in ['int64', 'float64']:
        data[col] = data[col].fillna(data[col].mean())
    else:
        data[col] = data[col].fillna(data[col].mode()[0])

# -----
##Detect & Remove Salary Outliers (IQR)
# -----
# Auto detect salary column
salary_col = None
for col in data.columns:
    if 'salary' in col.lower():
        salary_col = col
        break

Q1 = data[salary_col].quantile(0.25)
Q3 = data[salary_col].quantile(0.75)
IOR = Q3 - Q1

lower= Q1 - 1.5 * IOR
upper= Q3 + 1.5 * IOR

data= data[(data[salary_col] >= lower) & (data[salary_col] <= upper)]

# -----
##Standardize Department Names
# -----
for col in data.columns:
    if 'department' in col.lower():
        data[col] = data[col].str.lower()

##-----
# 4. Label Encoding
# -----
le= LabelEncoder()

for col in data.columns:
    if data[col].dtype == 'object':
        data[col] = le.fit_transform(data[col])

# -----
##Final Output
# -----
print("\nProcessed Data:")
print(data.head())

# Save file
data.to_csv("processed_employee_data.csv", index=False)
```

Ouput:

Original Data:

	ID	Experience_Years	Age	Gender	Salary
0	1	5	28	Female	250000
1	2	1	21	Male	50000
2	3	3	23	Female	170000
3	4	2	22	Male	25000
4	5	1	17	Male	10000

Columns: Index(['ID', 'Experience_Years', 'Age', 'Gender', 'Salary'], dtype='object')

Processed Data:

	ID	Experience_Years	Age	Gender	Salary
0	1	5	28	0	250000
1	2	1	21	1	50000
2	3	3	23	0	170000
3	4	2	22		25000
4	5	1	17		10000